

DataShield User Guide

What DataShield Is

DataShield protects small files for transfer over unreliable channels: radio, text chats, e-mail, damaged media. A file is encoded into a stream of short self-contained packets (75 bytes = 100 Base64 characters), and the receiver reconstructs it even when some packets are lost, duplicated, or buried in noise. The codec works both ways: encoding adds redundancy, decoding accumulates and reassembles the original file.

Requirements

- .NET 10 (Desktop Runtime) or a self-contained build of the application.
- Operating system: Windows, Linux, macOS (the GUI is built on Avalonia).
- The algorithms need no network and run fully locally.

Preparing a File for Transfer (Encoding)

1. Choose a file. Limits: size up to 4 MB (more precisely, up to $(65,535 - M) \cdot 64$ bytes with ECC; the size field allows up to 16,777,215 bytes, but the volume limit kicks in earlier); long names are automatically shortened to 14 characters with the extension preserved (e.g., `documents.tar.gz` → `docume~.tar.gz`).
2. Set the ECC redundancy, % — how many redundant packets to add to the data packets. The default is 10%: recovery is guaranteed while no more than 10% of sectors are lost. A higher percentage means more reliability but a longer stream.
3. Choose the output format:
 - **Text (Base64)** — one packet per line; fits chats, e-mails, text files;
 - **Binary** — raw 75-byte packets; more compact, for file-based channels.
4. Press "Encode". Progress shows the phases: data preparation → ECC computation → packet building. The result is a stream ready to copy; you may copy it whole or in parts.

The encoder automatically inserts several header copies (start, end, and evenly across the stream, ~3%), so the file's properties appear in the stream even under heavy losses.

Receiving and Recovering (Decoding)

1. Paste the received Base64 text or load a binary file; you may feed the stream in parts and in any order — the scanner finds packets in noise and retains everything not yet recognized.
2. While scanning, the UI shows: found files (headers), each file's sector map (received/missing volumes), confirmation and collision counters.

3. Press "Assemble". If the data suffices (all sectors present, or losses covered by ECC), the file is restored and verified by SHA-256 — the result is guaranteed bit-perfect. If the data is insufficient, the decoder refuses honestly: collect more packets and try again.

Good to know:

- Packets may arrive in any order and with repeats — duplicate copies merely raise the confirmation counters.
- A stream may contain several files interleaved — each is assembled by its own header.
- Sectors that arrive before their file's header are not lost: the decoder rebinds them once the header appears.

Recovery Guarantees

- Up to M sectors lost ($M \approx N \cdot \text{ECC\%}$): the file is restored by the Reed–Solomon code.
- All packets present: the file is assembled directly.
- Forged packets (foreign data with a valid hash is practically impossible): when versions look equally plausible, the decoder searches through combinations and accepts only the one whose SHA-256 matches the header.
- Integrity control: SHA-256 in the header; a result without a matching hash is never produced.

Demonstration and Self-Check

The bundled console bench (`DataShield.Demo`) continuously generates random files, damages them randomly, and verifies recovery. Indicators: **PASS** — restored bit-perfect; **WARN** — the damage deliberately exceeds the safety margin, a refusal is expected; **FAIL** — a codec failure (does not occur in normal operation). Keeping the bench open is a convenient way to watch stability and speed.

Quick Reference

Parameter	Value
Packet	75 bytes = 100 Base64 characters
Payload	64 bytes per packet
Default ECC	10% (recovers up to 10% losses)
Header copies	~3%, minimum 3 (start/middle/end)
Maximum file	~4 MB without ECC, less with ECC
File name	ASCII, up to 14 characters (longer — with <code>~</code>)
Verification	SHA-256 + truncated SHA-256 per packet